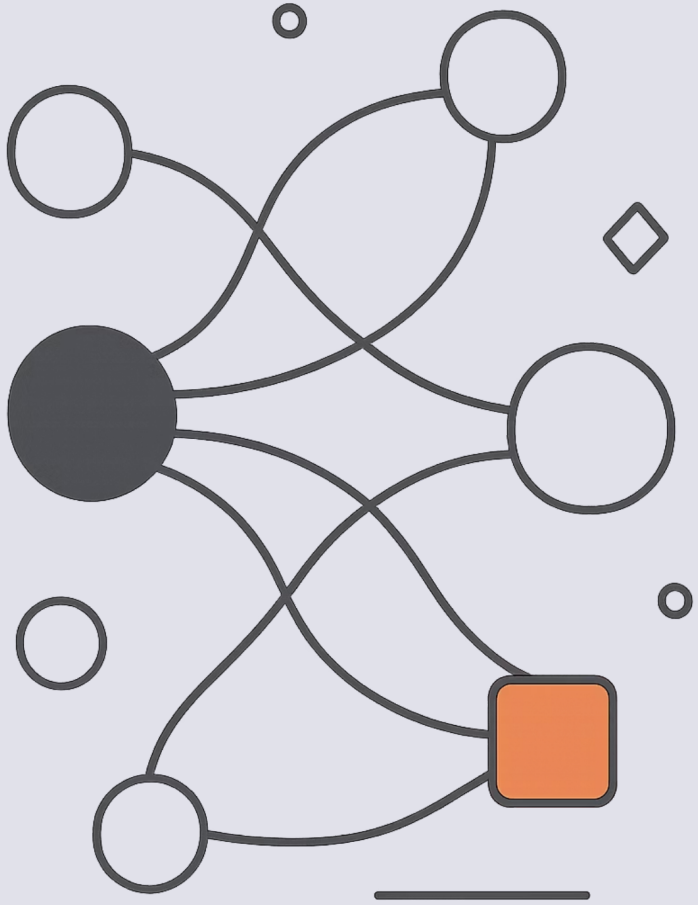
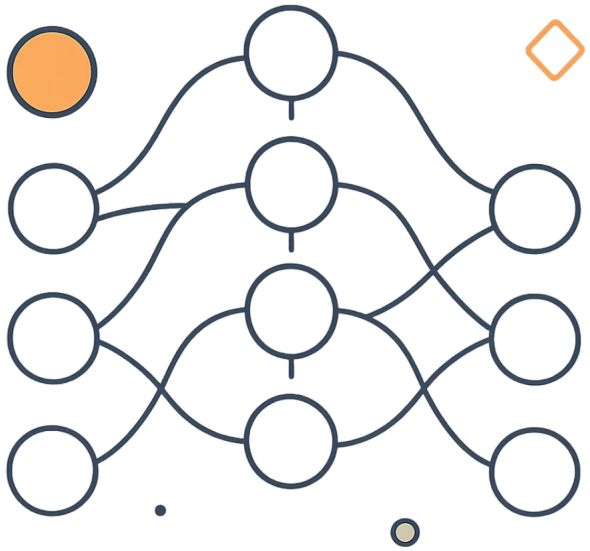
Deep Learning: From Research to Production

A practical journey through the tools, architectures, and pipelines that transform a deep learning experiment into a reliable, scalable production system.

DEEP LEARNING PYTORCH TRANSFORMERS MLOPS DOCKER



Deep Learning Fundamentals



What is Deep Learning?

A subset of machine learning that uses multi-layer neural networks to automatically learn patterns from large volumes of data — without hand-crafted features.

Key insight: Deep learning automatically learns useful representations from raw data, eliminating manual feature engineering.

- Images
- Text
- Speech
- Video

PyTorch

PyTorch is the dominant framework for deep learning research and production, offering dynamic computation graphs, seamless GPU support, and a rich ecosystem.

Dataset & DataLoaders

Load, transform, and batch data efficiently for training loops.

Model & Loss

Define architectures with `nn.Module` and compute gradients via a loss function.

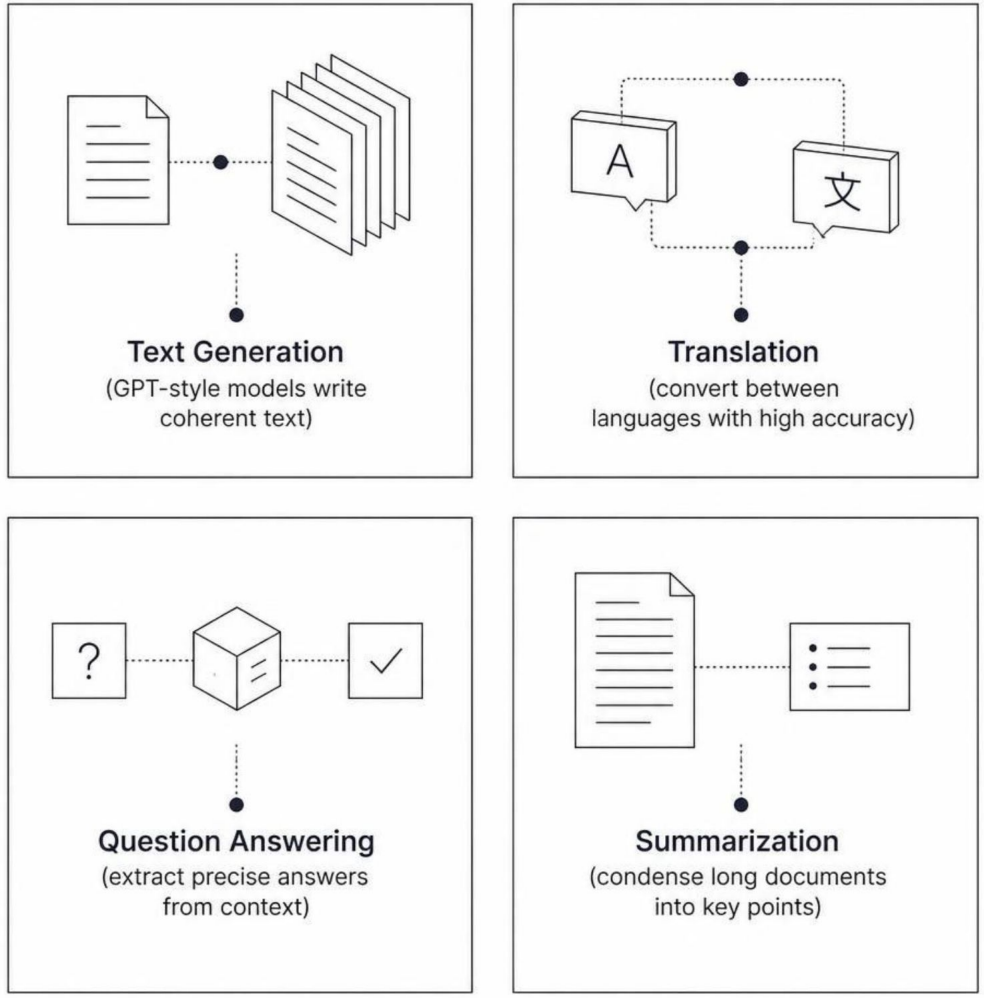
Backpropagation & Optimizer

Update weights using `loss.backward()` and an optimizer like Adam or SGD.

Trained Model

Export via TorchScript or ONNX for inference serving.

Transformers



What are Transformers?

A neural architecture built on**self-attention**, enabling models to capture long-range dependencies in sequences with high parallelism.

Self-Attention

Weighs the importance of every token relative to every other.

Parallelizable

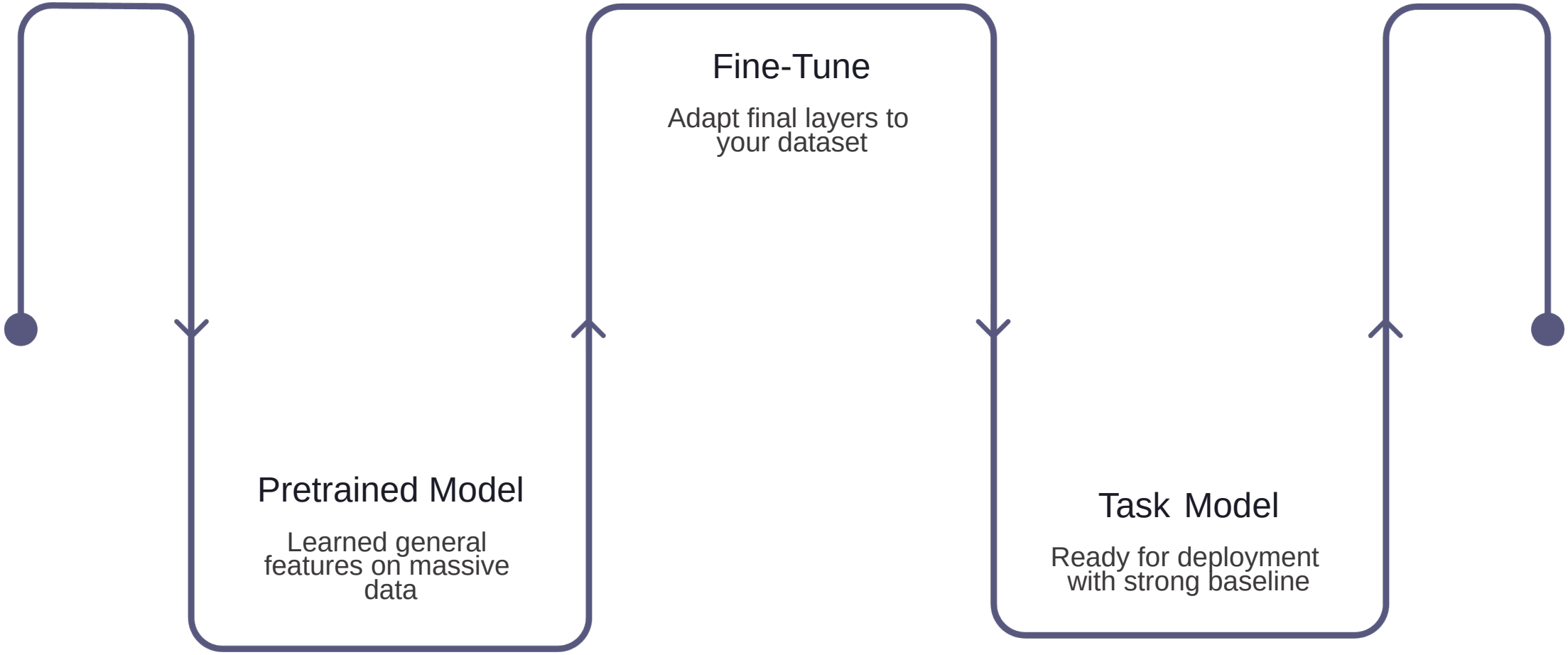
Processes entire sequences simultaneously, unlike RNNs.

Foundation Models

GPT, BERT, and T5 all derive from this architecture.

Transfer Learning & Hugging Face

Instead of training from scratch, start from a pretrained model and fine-tune on your specific task — saving time, data, and compute.



Why Transfer Learning?

- Less Data
- Faster Training
- Lower Compute

Hugging Face Ecosystem

Thousands of pretrained models across NLP, vision, and audio
Unified tokenizers and dataset libraries
Model Hub for sharing and versioning
`transformers` and `accelerate` for fine-tuning

Distributed Training

Modern models are too large for a single GPU. Distributed strategies split the workload across devices to train faster and at greater scale.

Data Parallelism

Split batches across GPUs; each holds a full model copy.

Model Parallelism

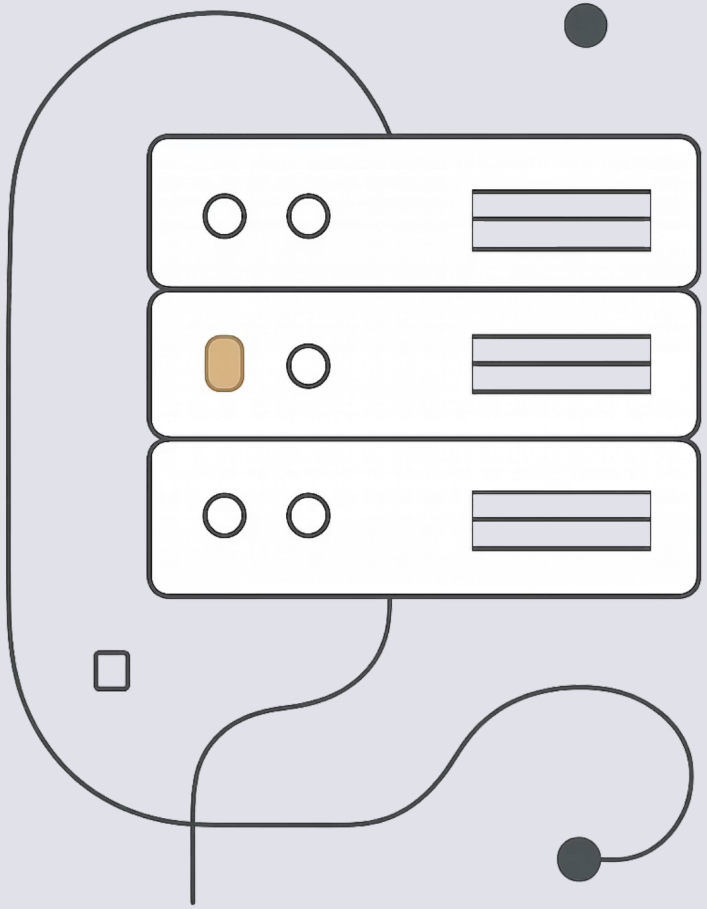
Split model layers across GPUs for models that don't fit in memory.

Pipeline Parallelism

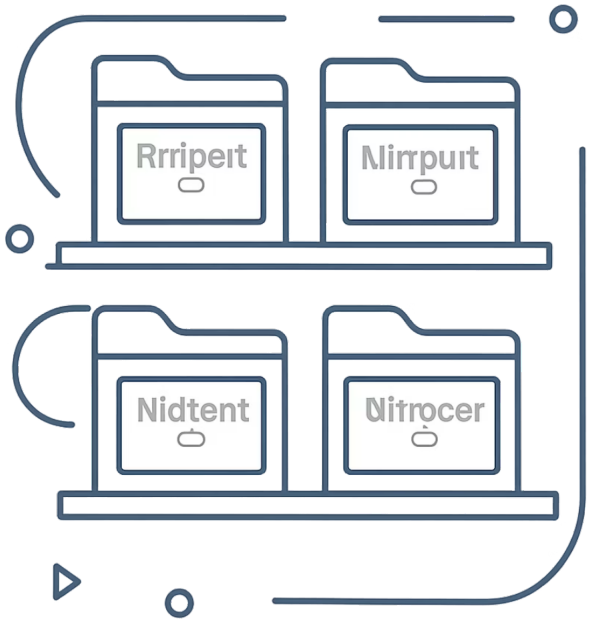
Divide the model into stages processed sequentially across devices.

FSDP / ZeRO

Shard optimizer states and gradients to dramatically reduce memory footprint.



Model Registry



What is a Model Registry?

A centralized system to**store, version, and govern**ML models throughout their lifecycle — from experiment to production.

Flow: Train → Evaluate → Register → Approve → Deploy

Version

Track model, dataset, and code commit together.

Metrics

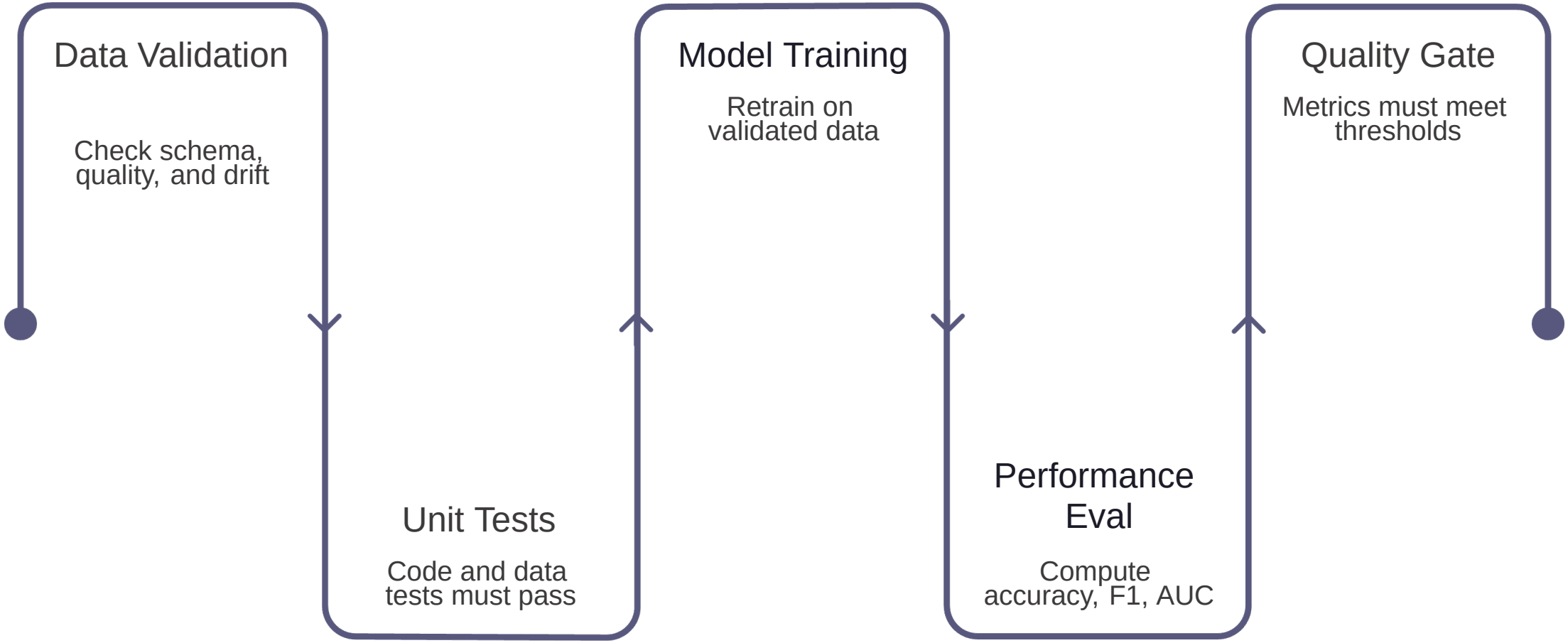
Store evaluation scores, hyperparameters, and owner info.

Governance

Approval workflows, deployment status, and rollback capability.

CI/CD for Machine Learning

ML pipelines extend traditional CI/CD by adding data validation, model training, and performance gates before any deployment proceeds.



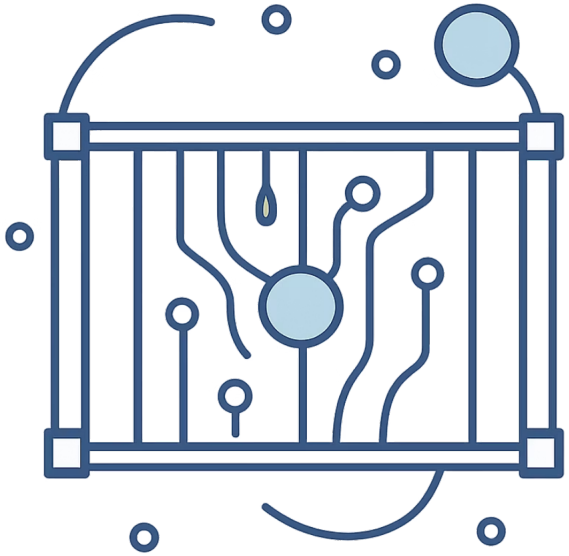
Quality Gate Examples

Accuracy ≥ 90%	F1 ≥ 85%
AUC ≥ 90%	

Key Difference from Traditional CI/CD

Standard pipelines test code only. ML CI/CD must also validate **data quality** and **model performance** — a failing metric blocks deployment automatically.

Docker for ML



Solving "It Works on My Machine"

Docker packages your application, Python runtime, PyTorch, dependencies, and model artifacts into a single portable**container**— eliminating environment drift between development and production. Images are stored in**Docker Hub**,**Amazon ECR**, or a private registry.

1

Write Dockerfile

Define base image, dependencies, and entry point.

2

Build Docker Image

Creates a reproducible snapshot of the environment.

3

Run Container

Deploy consistently across any host or cloud.

End-to-End Deep Learning Lifecycle

Production AI is not a single model — it is an**automated, governed lifecycle**connecting every stage from raw data to continuous improvement.

1

Data

Collect, clean, and version datasets.

2

Deep Learning & PyTorch

Design and train neural network models.

3

Transformers

Apply self-attention architectures for NLP, vision, and multimodal tasks.

4

Distributed Training

Scale across multiple GPUs for large models.

5

Model Evaluation

Measure accuracy, F1, AUC, and other metrics.

6

Model Registry

Version, track, and govern model artifacts.

7

CI/CD

Automate testing, quality gates, and deployment pipelines.

8

Docker

Package models into reproducible containers.

9

Deployment

Serve models to production environments.

10

Monitoring & Continuous Improvement

Track drift, retrain, and iterate.

Takeaway: Every tool in this stack serves a purpose — together they form a reliable, scalable path from experiment to production AI.